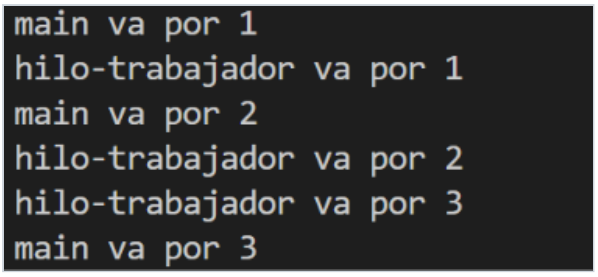
Cuaderno de actividades

PSP-UD2 · RA2

Actividad 1 · Del proceso al hilo: crear, esperar y parar

Actividad 1.1 · start() frente a run()

CAPTURA · SALIDAS DE LA TERMINAL CON START() Y CON RUN()



--- con start() ---



--- con run() ---

4 · DIFERENCIA ENTRE START() Y RUN() CITANDO LAS DOS SALIDAS

Con `start()` hay dos hilos trabajando a la vez.

- La salida sale entrelazada: `main va por 1`, `hilo-trabajador va por 1`, `main va por 2`...
- `start()` pone en marcha el hilo-trabajador, que ejecuta la tarea mientras main ejecuta la suya con `tarea.run()`.
- Por eso unas líneas dicen hilo-trabajador y otras main: cada hilo que ejecuta la tarea imprime su propio nombre.

Con `run()` solo trabaja main.

- Todas las líneas dicen main y salen seguidas, sin mezclarse: `main va por 1, 2, 3`, dos veces.
- `run()` no crea ningún hilo. Es una llamada a un método normal, y la ejecuta quien la escribe: main.
- El hilo-trabajador se ha creado, pero nunca se ha arrancado. Por eso `join()` vuelve al momento y el mensaje `main: el trabajador ha terminado` no es cierto.

Conclusión: `start()` crea un hilo nuevo que ejecuta la tarea en paralelo; `run()` solo ejecuta la tarea en el hilo que ya está corriendo.

Actividad 1.1 · Pasos 5 y 6 · tres hilos sobre la misma tarea

CAPTURA · LA SALIDA: LA ÚLTIMA LÍNEA DEL MAIN SALE LA ÚLTIMA

```
hilo1 va por 1
hilo3 va por 1
hilo2 va por 1
hilo3 va por 2
hilo1 va por 2
hilo2 va por 2
hilo3 va por 3
hilo2 va por 3
hilo1 va por 3
main: las 3 tareas han terminado.
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2>
```

Actividad 1.3 · El reparto de la UD1, ahora entre hilos

9 · MEDIDAS CON N = 1, 2, 4 Y 8 (CALENTAMIENTO DESCARTADO, CINCO PASADAS, MEDIANA)

Portátil de 4 núcleos, con el equipo en reposo.

```
TOTAL 1 hilos: 25938 ms total=1270607
TOTAL 1 hilos: 25973 ms total=1270607
TOTAL 1 hilos: 25899 ms total=1270607
TOTAL 1 hilos: 25980 ms total=1270607
TOTAL 1 hilos: 25797 ms total=1270607
TOTAL 2 hilos: 16598 ms total=1268420
TOTAL 2 hilos: 16424 ms total=1268435
TOTAL 2 hilos: 16516 ms total=1268366
TOTAL 2 hilos: 16369 ms total=1268376
TOTAL 2 hilos: 16400 ms total=1268381
TOTAL 4 hilos: 8927 ms total=1264155
TOTAL 4 hilos: 8927 ms total=1264183
TOTAL 4 hilos: 8838 ms total=1264142
TOTAL 4 hilos: 8880 ms total=1264150
TOTAL 4 hilos: 8938 ms total=1264170
TOTAL 8 hilos: 7199 ms total=1263523
TOTAL 8 hilos: 7171 ms total=1263459
```

```
TOTAL 8 hilos: 7199 ms total=1263523
TOTAL 8 hilos: 7171 ms total=1263459
TOTAL 8 hilos: 7171 ms total=1263291
TOTAL 8 hilos: 7174 ms total=1263302
TOTAL 8 hilos: 7726 ms total=1263625
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a1-hilos>
```

N hilos	Pasadas (ms)	Mediana (ms)	total en la mediana	Primos que faltan
1	25938 · 25973 · 25899 · 25980 · 25797	25938	1270607	0
2	16598 · 16424 · 16516 · 16369 · 16400	16424	1268435	2172
4	8927 · 8927 · 8838 · 8880 · 8938	8927	1264155	6452
8	7199 · 7171 · 7171 · 7174 · 7726	7174	1263302	7305

10 · MEDIDAS DE HILOS AL LADO DE LAS DE PROCESOS DE LA UD1 (MISMO RANGO)

N	Procesos UD1 · mediana (ms)	Hilos · mediana (ms)	Hilos · total de primos
1	28010	25938	1270607
2	17473	16424	1268435
4	10698	8927	1264155
8	10585	7174	1263302

11 · EQUIPO Y CONDICIONES DE LA MEDIDA

- **Equipo:** portátil HP Envy x360, Windows 11 Home, 7,39 GB de RAM, 4 núcleos.
- **Java:** 21.0.12 LTS (el mismo que en las medidas de procesos de la UD1).
- **Rango:** [2, 20.000.000), el mismo de la UD1.
- **Ejecución:** desde la terminal de VS Code con `java PrimosHilos.java N`, para N = 1, 2, 4 y 8.
- **Método:** una pasada de calentamiento descartada, cinco pasadas por cada N y me quedo con la mediana.
- **Condiciones:** equipo en reposo, con el navegador y el resto de programas cerrados.

12 · ¿CUÁL ARRANCA ANTES, EL REPARTO ENTRE PROCESOS O ENTRE HILOS? ¿CUÁL ESCALA MEJOR?

Arranca antes el reparto entre hilos. Crear un hilo es barato porque vive dentro del mismo proceso y comparte su memoria; un proceso tiene que arrancar una JVM nueva. Se ve con N=1: 25.938 ms con hilos frente a 28.010 ms con procesos.

Escalan mejor los hilos: con 4 mejoran 2,91x (procesos 2,62x) y con 8 mejoran 3,62x (procesos 2,65x, prácticamente igual que con 4, porque 8 JVM compiten por 4 núcleos y la memoria). Pero los hilos dan un total incorrecto (con 8 faltan 7.305 primos), así que de momento son más rápidos dando un resultado que está mal.

13 · ¿POR QUÉ EL TROZO 0 TERMINA SIEMPRE MUCHO ANTES QUE EL ÚLTIMO?

Porque los trozos tienen la misma cantidad de números, pero no el mismo trabajo. `esPrimo(x)` prueba divisores hasta la raíz de x: con los números pequeños del trozo 0 termina enseguida, y con los números grandes del último trozo (cerca de 20.000.000) tiene que probar muchos más. Repartir el rango en partes iguales no reparte la carga por igual: el trozo 0 acaba pronto y su núcleo se queda sin nada que hacer, mientras el último sigue trabajando, y es ese trozo el que marca el tiempo total.

Actividad 1.4 · El servicio del club y su limpiador

16 · CÓDIGO: LAS LÍNEAS DEL MAIN QUE PARAN EL LIMPIADOR Y LO ESPERAN

```
Thread.sleep(args.length > 0 ? Long.parseLong(args[0]) : 3000);
//Actividad 1.4 Paso 16
limpiador.interrupt(); //tras dejar correrlo 3s lo interrumpimos
limpiador.join(); //lo esperamos.
```

16 · CAPTURA: LA TRAZA DEL ARRANQUE Y LA PARADA DEL SERVICIO

```
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a1-hilos> java ServicioClub.java
13:17:46.185 [main] servicio del club arrancado
13:17:46.242 [limpiador] vuelta de limpieza: empiezo
13:17:47.120 [limpiador] franja 5 liberada (era de bruno)
13:17:47.532 [limpiador] vuelta de limpieza: termino sin dejar nada a medias
13:17:48.535 [limpiador] vuelta de limpieza: empiezo
13:17:48.753 [limpiador] franja 2 liberada (era de ana)
13:17:49.306 [limpiador] vuelta de limpieza: termino sin dejar nada a medias
13:17:49.307 [limpiador] limpiador detenido
13:17:49.307 [main] main termina
```

Actividad 1.5 · Qué lleva hilo y qué no

17 · CLASIFICACIÓN DE LAS SEIS OPERACIONES DEL SERVICIO

Operación del club	A qué espera	Grupo	¿Lleva hilo? ¿Por qué?
Comprobar si una franja está libre	A nada: es leer una posición de memoria	Instantánea	**No.** No espera a nada; el hilo costaría más que la operación.
Liberar las franjas caducadas cada segundo	Al reloj	Espera mucho	**Sí.** Espera al reloj; duerme en segundo plano sin bloquear el servicio.
Enviar el correo de confirmación de una reserva	A un servidor de correo	Espera mucho	**Sí.** Espera al servidor de correo; así el usuario no espera con él.
Calcular cuántos primos hay en el rango común	A nada: es CPU pura, pero se trocea	Calcula mucho	**Sí.** CPU pura: se reparte entre hilos, uno por núcleo.
Formatear la fecha de una reserva para la pantalla	A nada: microsegundos	Instantánea	**No.** Microsegundos; no espera a nada.
Escribir el registro del día en un fichero	Al disco	Espera mucho	**Sí.** Espera al disco; el servicio sigue mientras se escribe.

18 · OPERACIONES QUE NO LLEVAN HILO: QUÉ SE GANARÍA Y POR QUÉ NO COMPENSA

Comprobar si una franja está libre

- **Qué se ganaría:** nada. Es leer una posición de memoria y acaba al instante.
- **Por qué no compensa:** crear y arrancar un hilo cuesta mucho más que la propia lectura, y habría que esperarlo con `join()` para saber la respuesta.
- **Además:** otro hilo más tocando `franja[]` agrava la carrera de la sección crítica.

Formatear la fecha de una reserva para la pantalla

- **Qué se ganaría:** nada. Tarda microsegundos y no espera a nada.
- **Por qué no compensa:** el hilo tardaría más en crearse que la operación en hacerse.
- **Además:** la pantalla necesita el resultado ya, así que habría que esperarlo igual.

Actividad 2 · La carrera, demostrada con un número

Actividad 2.1 · Las predicciones

1 · (A) DOS HILOS INCREMENTAN UN CONTADOR UN MILLÓN DE VECES CADA UNO. ¿QUÉ SALE?

El contador incrementado 2 millones de veces: 2.000.000.

2 · (B) EN MIL INTENTOS DE RESERVAR LA MISMA FRANJA CON DOS HILOS, ¿CUÁNTAS VECES SE CONFIRMARÁN LAS DOS?

Cada vez: las 1.000 veces.

Actividad 2.2 · El contador

4 · CAPTURA: LAS CINCO EJECUCIONES DE CONTADOR.JAVA

```
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1572517  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1373856  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1742444  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1378968  esperado: 2000000
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2\a2-carrera> java Contador.java
Obtenido: 1807670  esperado: 2000000
```

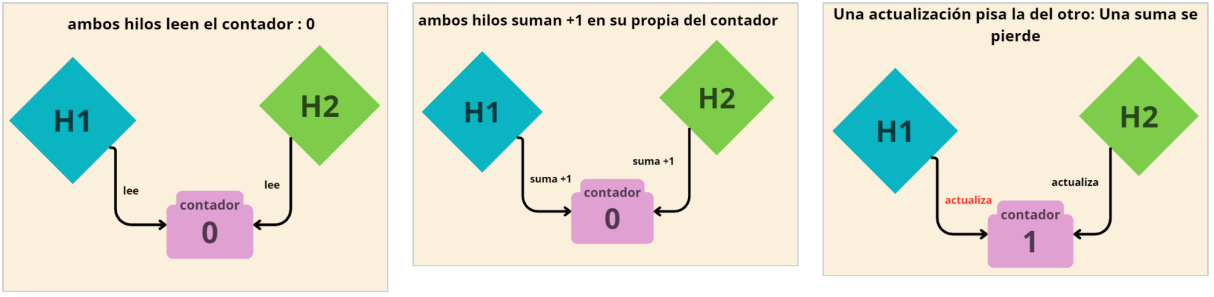
5 · ¿POR QUÉ SALEN DISTINTOS? LOS TRES PASOS DE N++

Salen distintos porque `contador++` no es una sola operación, son tres pasos:

1. **Leer** el valor de `contador`.
2. **Sumar** 1 a ese valor.
3. **Escribir** el resultado en `contador`.

Los dos hilos ejecutan estos pasos a la vez y pueden mezclarse: si los dos leen el mismo valor antes de que el otro escriba, los dos escriben el mismo resultado y se pierde un incremento. Cuántas veces pasa depende de cómo reparta el sistema operativo el procesador entre los hilos, que cambia en cada ejecución. Por eso cada vez se pierden incrementos distintos y el número final es diferente.

6 · EL ENTRELAZADO DE LOS DOS HILOS QUE PIERDE UN INCREMENTO



8 · ¿CUÁNTOS PRIMOS SE PERDIERON CON 2, 4 Y 8 HILOS? ¿POR QUÉ CON UNO SOLO NO SE PERDÍA NINGUNO?

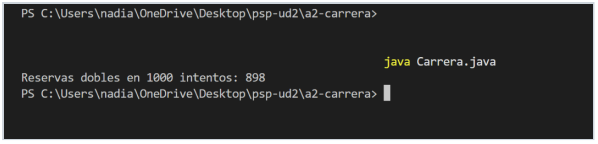
N hilos	total	Primos perdidos
1	1270607	0
2	1268435	2172
4	1264155	6452
8	1263302	7305

Con 2 hilos se perdieron 2.172 primos, con 4 se perdieron 6.452 y con 8 se perdieron 7.305 (frente a los 1.270.607 que da con 1 hilo).

Con un solo hilo no se pierde ninguno porque nadie más toca `total`: ese hilo lee, suma y escribe sin que otro se meta en medio. Con varios hilos, todos hacen `total++` sobre la misma variable compartida, y cuando dos leen el mismo valor a la vez, uno de los incrementos se pisa. Cuantos más hilos, más veces coinciden y más primos se pierden.

Actividad 2.3 · La reserva: mil intentos

10 Y 11 · CAPTURA: LAS DOS EJECUCIONES, CON YIELD Y SIN YIELD



--- con Thread.yield() ---



--- sin Thread.yield() ---

12 · DOBLES EN MIL INTENTOS CON YIELD Y SIN YIELD

Con `yield`: 898 dobles de 1000. Sin `yield`: 647 dobles de 1000.

13 · CONCLUSIÓN CON ESOS DOS NÚMEROS

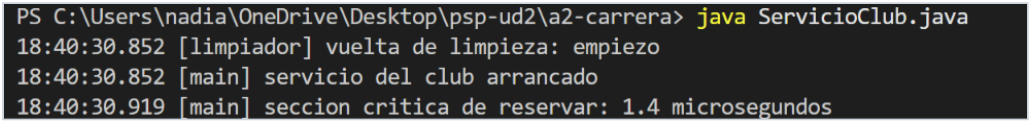
El fallo está en los dos casos. El `yield()` no lo crea: solo ensancha el hueco entre comprobar y actuar, y por eso con él salen más dobles. Sin `yield` el hueco sigue existiendo, es más estrecho, pero aun así se confirmaron dos reservas 647 veces de 1000. Que sea menos probable no significa que no pase: con mil usuarios reservando de verdad, un fallo poco probable acaba pasando seguro.

Actividad 2.5 · La sección crítica y el inventario

14 A 16 · CÓDIGO: EL MÉTODO RESERVAR CON LA SECCIÓN CRÍTICA MARCADA

```
static boolean reservar(int id, String quien) {
    long t0 = System.nanoTime();
    // SECCIÓN CRÍTICA: empieza aquí
    if (franja[id] == null) { // 1 · comprobar
        franja[id] = quien; // 2 · actuar
        caduca[id] = System.currentTimeMillis() + 900_000; // quince
minutos para pagar
        // SECCIÓN CRÍTICA: acaba aquí
        long t1 = System.nanoTime();
        traza("seccion critica de reservar: " + (t1 - t0) / 1000.0 + "
microsegundos");
        return true;
    }
    return false;
}
```

Lo que queda dentro de la sección crítica tarda **1,4 microsegundos**:



17 · INVENTARIO: MÉTODOS DEL SERVICIO QUE TOCAN EL MISMO ESTADO COMPARTIDO

Método	Qué toca	¿Sección crítica?
`reservar(franja, usuario)`	Escribe el ocupante de una franja	Sí
`cancelar(franja)`	Borra el ocupante de una franja	Sí
`limpiar()` (el limpiador de la actividad 1)	Libera las franjas caducadas	Sí
`estaLibre(franja)`	Solo lee	No
`informeDelDia()`	Recorre las seis franjas y las cuenta	Sí

18 · ¿CUÁNTO TARDA LO QUE HABÉIS DEJADO DENTRO? EN MICROSEGUNDOS

1,4 microsegundos.

19 · LA SECCIÓN CRÍTICA DEL CLUB

Estado compartido: los arrays `franja[]` (quién ocupa cada franja) y `caduca[]` (cuándo caduca cada reserva). Son `static`: una sola copia para todos los hilos.

Métodos que lo tocan: `reservar`, `cancelar`, `limpiar` e `informeDelDia`.

Qué pasa si dos hilos entran a la vez: en `reservar`, los dos comprueban `franja[id] == null` antes de que ninguno escriba, los dos ven la franja libre y los dos reservan. Dos usuarios reciben «Reserva confirmada» para el mismo barco. No sale ningún error.

Los tres números que lo demuestran:

- **Contador:** 1.572.517 · 1.373.856 · 1.742.444 · 1.378.968 · 1.807.670 (esperado 2.000.000).
- **Primos perdidos:** 2.172 con 2 hilos, 6.452 con 4 y 7.305 con 8.
- **Reservas dobles:** 898 de 1000 con `yield`, 647 de 1000 sin él.

Actividad 3 · Que solo pase uno: tres soluciones medidas

VOTO ANTES DE ESCRIBIR CÓDIGO (A BLOQUEO, B ATÓMICA O C COLECCIÓN CONCURRENTE)

C

Actividad 3.1 · Tres versiones de la reserva

1 A 4 · CÓDIGO: LOS CUATRO MÉTODOS DE RESERVA (A, B, C Y EL DE GET + PUT)

```
private static final Object CERROJO = new Object();
static final AtomicReference<String> franja = new AtomicReference<>();
static final ConcurrentHashMap<Integer,String> mapa = new
ConcurrentHashMap<>();

// Versión A · bloqueo
static void reservarA(String quien) {
    synchronized(CERROJO)
    {
        if (usuario == null) { // 1 · comprobar
            Thread.yield(); // 2 · la ventana, ensanchada a propósito
            usuario = quien; // 3 · actuar
            confirmadas++;
        }
    }
}

// Versión B · atómica
static void reservarB(String quien) {
    if(franja.compareAndSet(null, quien))
    {
        confirmadas++;
    }
}

// Versión C · colección concurrente
static void reservarC(String quien) {
    if(mapa.putIfAbsent(1, quien)==null)
    {
        confirmadas++;
    }
}

// La que parece correcta y no lo es: get + put
static void reservarMal(String quien) {
    if (mapa.get(1) == null)
    {
        Thread.yield();
        mapa.put(1, quien);
        confirmadas++;
    }
}
```

Actividad 3.2 · La misma prueba para las cuatro

5 Y 6 · CAPTURA: LAS CUATRO SALIDAS DE LA PRUEBA Y LA DE LA VERSIÓN SIN PROTECCIÓN

```
A · bloqueo:     0 dobles en 1000 intentos
B · atómica:     0 dobles en 1000 intentos
C · colección:   0 dobles en 1000 intentos
Mal · get + put: 917 dobles en 1000 intentos
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2>
```

```
--- A, B y C protegidas ---
```

```
A · bloqueo:     797 dobles en 1000 intentos
B · atómica:     0 dobles en 1000 intentos
C · colección:   0 dobles en 1000 intentos
Mal · get + put: 898 dobles en 1000 intentos
PS C:\Users\nadia\OneDrive\Desktop\psp-ud2>
```

```
--- A sin synchronized: vuelve a fallar ---
```

7 · ¿A QUÉ VERSIÓN LE QUITASTEIS LA PROTECCIÓN Y CUÁNTOS DOBLES DIO?

A la versión A (bloqueo): sin el `synchronized` dio 797 dobles en 1000 intentos.

8 · ¿POR QUÉ PUTIFABSENT SÍ Y GET SEGUIDO DE PUT NO? CON EL NÚMERO DE LA PRUEBA

`putIfAbsent` hace comprobar y actuar en una sola operación atómica: mira si la franja está vacía y, si lo está, mete al usuario, sin que ningún otro hilo pueda meterse en medio. Por eso solo gana uno: 0 dobles en 1000 intentos.

`get` seguido de `put` son dos operaciones. Cada una es segura por separado, pero entre las dos hay un hueco: los dos hilos hacen `get`, los dos ven la franja vacía y los dos hacen `put`. Es el mismo comprobar y actuar de la actividad 2, aunque el mapa sea concurrente: 917 dobles en 1000 intentos.

Actividad 3.3 · El acumulador de primos

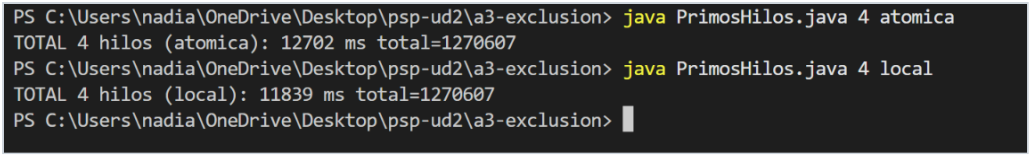
9 · CÓDIGO: LAS DOS VERSIONES DEL CUERPO DEL HILO QUE CUENTA

```
static final AtomicLong total = new AtomicLong(0);

// Manera 1 · AtomicLong: cada primo encontrado se suma al AtomicLong
hilos[i] = new Thread(() -> {
    for (long x = a; x < b; x++) if (esPrimo(x)) total.incrementAndGet();
}, "trozo-" + i);

// Manera 2 · acumulador local: cada hilo cuenta en su variable y suma una
sola vez al final
hilos[i] = new Thread(() -> {
    long local = 0;
    for (long x = a; x < b; x++) if (esPrimo(x)) local++;   // local: sin
carrera
    total.addAndGet(local);                                  // una sola
suma por hilo, y atómica
}, "trozo-" + i);
```

10 · CAPTURA: LAS DOS EJECUCIONES, CON SU TOTAL Y SU TIEMPO



11 · ¿SE NOTA LA DIFERENCIA DE TIEMPO ENTRE LAS DOS? ¿POR QUÉ?

Sí, aunque no mucho: con 4 hilos, el `AtomicLong` tardó 12.702 ms y el acumulador local 11.839 ms, unos 900 ms menos.

Con el `AtomicLong`, cada primo encontrado es una operación atómica sobre la misma variable compartida: más de 1,2 millones de veces, con los 4 hilos peleándose por ella. Cada vez que uno la cambia, los demás núcleos tienen que volver a leerla, y eso frena. Con el acumulador local, cada hilo cuenta en su propia variable sin molestar a nadie y solo toca la compartida una vez al final: 4 operaciones atómicas en total en vez de 1,2 millones.

La diferencia no es mayor porque la mayor parte del tiempo se va en `esPrimo`, no en sumar.